Mini Project Report

On

“SkillNest: Full-Stack Technical Interview & Placement Preparation Platform”

By

Aditya Narwade
(PRN: 1272250292)

For

MCA40130 Mini Project

SY MCA

AY 2026-2027

Under The Guidance Of

Mrs. Sukanya Akolkar

Submitted To

In partial fulfillment of the requirement for the award of Degree of

Master of Computer Applications

Submitted Through

MIT-WPU

School of Computer Science and Engineering
Department of Computer Science and Applications

Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Department of Computer Science and Applications

Certificate

This is to certify that, **Aditya Narwade (PRN: 1272250292)**, student of **SY MCA, Semester III** has completed Mini Project entitled **"SkillNest: Full-Stack Technical Interview & Placement Preparation Platform"** under guidance of **Mrs. Sukanya Akolkar** in partial fulfilment of requirement for the award of **Master of Computer Applications** prescribed under **Dr. Vishwanath Karad MIT World Peace University, Pune**.

This project is the record of authentic work carried out by him out during the academic year **2026-2027**.

Mrs. Sukanya Akolkar
Internal Guide

Dr. P. S. Metkewar
Program Head- MCA

Dr. Anuradha S. Kanade
Program Director

Date: / / 2026

Acknowledgement

I wish to express our profound gratitude to all the individuals who supported us throughout the completion of our Mini Project III. This report is a testament to their guidance, patience, and resources.

We are deeply indebted to our esteemed project guide, **Mrs. Sukanya Akolkar**, for their invaluable mentorship, direction, and encouragement. Their advice was crucial in defining the scope and methodology of this project.

We extend our sincere thanks to the Head of the Program, **Dr. P. S. Metkewar**, and the Program Director, **Dr. Anuradha S. Kanade**, for providing the necessary facilities, academic framework, and environment conducive to advanced project work and research.

Our appreciation also goes to all the Faculty Members of the **Department of Computer Science and Applications** for their comprehensive instruction and expertise, which equipped us with the knowledge foundational to this project.

We thank our fellow Project Group Members, for their unwavering dedication, collaborative spirit, and commitment to shared goals. This project is a product of our collective effort.

Finally, we are grateful to our Classmates for their constructive feedback and discussions, which enriched our understanding and helped us refine our approach.

Signature:	
Name:	**Aditya Narwade**
PRN:	1272250292
Place:	Pune

Date: / / 2026

Declaration

I, **Aditya Narwade**, on behalf of my team here by declare that this Mini Project report entitled **"SkillNest: Full-Stack Technical Interview & Placement Preparation Platform"** is an authentic record of our own work carried out under the supervision of **Mrs. Sukanya Akolkar**.

I further declare that the content of this report, including all design, coding, testing, and analysis, is original and has not been submitted previously for any other degree or diploma at this or any other institution.

I take sole and full responsibility for any errors, omissions, or misrepresentations found within this report.

Signature:	
Name:	**Aditya Narwade**
PRN:	1272250292
Place:	Pune

PROJECT REPORT INDEX

The table provided below lists all the required sections. Your final report must follow this exact sequence and numbering.

Index	Section Title	Page No.
1	**INTRODUCTION**	**6**
1.1	EXISTING SYSTEM, PROPOSED SYSTEM AND NEED AND OBJECTIVE OF SYSTEM	6
1.2	SCOPE OF WORK	7
1.3	OPERATING ENVIRONMENT – HARDWARE AND SOFTWARE	7
1.4	DESCRIPTION OF TECHNOLOGY USED	8
2	**PROPOSED SYSTEM**	**9**
2.1	FEASIBILITY STUDY	9
2.2	USER REQUIREMENTS	9
3	**ANALYSIS AND DESIGN (If Applicable)**	**11**
3.1	OBJECT DIAGRAM	11
3.2	CLASS DIAGRAM	11
3.3	USE CASE DIAGRAM	12
3.4	SEQUENCE DIAGRAM	12
3.5	ACTIVITY DIAGRAM	13
3.6	MODULE HIERARCHY DIAGRAM	14
3.7	USER INTERFACE DESIGN (SCREENS ETC.)	14
3.8	TABLE SPECIFICATIONS	14
4	**INPUT AND OUTPUT SCREENS**	**18**
5	**TEST PROCEDURES AND IMPLEMENTATION**	**23**
6	**SAMPLE CODE**	**26**
7	**LIMITATIONS**	**30**
8	**FUTURE SCOPE**	**31**
9	**CONCLUSION**	**32**
10	**REFERENCES**	**33**

1. INTRODUCTION

1.1 EXISTING SYSTEM, PROPOSED SYSTEM AND NEED AND OBJECTIVE OF SYSTEM

1.1.1 Analysis of the Existing System

Modern campus recruitment drives and corporate technical screenings place substantial weight on two complementary competencies: core conceptual multiple-choice question (MCQ) screening and hands-on coding ability. However, the preparation ecosystem available to computer science and application students is severely fragmented.

Currently, candidates navigate disparate platforms: algorithmic portals (LeetCode, HackerRank) that offer minimal structured conceptual MCQ preparation, static documentation repositories (W3Schools, MDN), and ad-hoc forums. Furthermore, academic institutions typically conduct mock technical tests using generic survey software such as Google Forms or basic Learning Management Systems (LMS). These ad-hoc setups suffer from critical vulnerabilities:

- **Absence of Anti-Cheat Controls:** Web-based quiz forms transmitting complete datasets allow candidates to easily inspect browser developer tools, examine JSON payloads, and extract correct answers prior to submission.
- **Delayed Evaluation & Feedback:** Students wait days for manual evaluation, severing the cognitive link between conceptual errors and accurate explanations.
- **Administrative Inefficiency:** Faculty coordinators must manually author quizzes, calculate cutoffs, and tabulate spreadsheets, consuming valuable academic resources.

1.1.2 Proposed System: SkillNest Architecture

SkillNest is an intelligent, full-stack technical interview preparation, automated testing, and placement calibration platform engineered to unify domain learning with rigorous assessment integrity. Developed upon a decoupled service-oriented architecture, SkillNest unites a modern React 19 single-page application (SPA) with an enterprise-grade Spring Boot 4.1.1 backend and a relational MySQL 8.0 database.

The system introduces three core architectural innovations:

- **Anti-Cheat Assessment Protocol:** Active tests communicate with a dedicated sanitization endpoint (`/api/quizzes/{id}/take`) that utilizes custom Data Transfer Objects (`TakeTestQuestionDTO`) to strictly omit correct answer indices and explanations from network payloads.
- **Automated Server-Side Evaluation Engine:** The backend `TestAttemptService` evaluates submissions against MySQL keys, calculates total scores, evaluates pass/fail criteria, and saves the attempt record atomically.
- **Automated HTML Scorecard Email Delivery:** Leveraging `JavaMailSender`, the platform compiles a responsive HTML scorecard summarizing test metrics and dispatches it directly to the student's registered inbox.

1.1.3 Need for the System

MCA candidates require a consolidated platform offering authentic technical screening questions, synchronized countdown timers, question navigation palettes, and immediate multi-dimensional feedback replicating genuine corporate hiring environments. SkillNest addresses this acute need by providing an industry-aligned assessment sandbox.

1.1.4 Primary Objectives of the System

- **Curated 9-Domain Knowledge Hub:** Seed and organize 108+ high-yield technical questions across Java, Python, C++, HTML, CSS, JavaScript, React, Node.js, and SQL.
- **Anti-Cheat Assessment Delivery:** Enforce server-side DTO sanitization preventing answer leakage via client inspection tools.

- **Real-Time Timed Assessment HUD:** Deliver an active test interface with live countdown timers, status-coded question palettes (Answered, Unanswered, Flagged), and auto-submission mechanisms.
- **Automated Server-Side Grading:** Calculate aggregate marks, percentage scores, and pass/fail thresholds against relational database keys.
- **Asynchronous Scorecard Email Dispatch:** Deliver styled HTML scorecards via Spring Boot's mail infrastructure.
- **Role-Based Administrative Control:** Supply comprehensive back-office interfaces for faculty staff to manage question banks and configure published quizzes.

1.2 SCOPE OF WORK

The scope of SkillNest spans end-to-end full-stack software development:

- User authentication and role guarding with 10-round salted BCrypt password hashing.
- Curated 9-domain technical knowledge repository with interactive self-paced solve modes.
- Timed assessment subsystem supporting custom durations, dynamic mark calculation, and draft/publish toggling.
- Interactive candidate test HUD with synchronized countdown timers, question navigation palettes, and modal confirmation dialogs.
- Transactional server-side grading engine with persistent attempt logging in MySQL.
- Administrative management suite for question bank CRUD, quiz authoring, and student attempt monitoring.

1.3 OPERATING ENVIRONMENT – HARDWARE AND SOFTWARE

Component	Minimum Specification	Recommended Specification
Client Processor	Dual-Core 1.8 GHz Intel / AMD / ARM	Quad-Core 2.5 GHz+ (Intel i5/i7/M-Series)
Client Memory (RAM)	4 GB RAM	8 GB - 16 GB RAM
Client Display	1280 x 720 (HD)	1920 x 1080 (FHD) or higher
Web Browser	Chromium 100+, Firefox 105+, Edge 100+	Latest Google Chrome, Microsoft Edge, Brave
Server Processor	2 vCPUs @ 2.0 GHz	4 vCPUs @ 2.8 GHz+
Server Memory (RAM)	4 GB RAM	8 GB - 16 GB RAM
Server Storage	10 GB SSD Storage	50 GB NVMe Storage
Operating System	Windows 10/11, Ubuntu 22.04 LTS, macOS 13+	Ubuntu 24.04 LTS / Windows Server 2022
Java Runtime	Java Development Kit (JDK) 21 LTS	Eclipse Temurin OpenJDK 21 LTS
Node.js Runtime	Node.js 18.x LTS	Node.js 20.x LTS or higher
Relational Database	MySQL Community Server 8.0	MySQL Server 8.0.35+

1.4 DESCRIPTION OF TECHNOLOGY USED

Technology	Verified Version	Architectural Role & Selection Rationale
Java	Version 21 LTS	Modern language features including records, pattern matching, and virtual threads for high backend concurrency.
Spring Boot	Version 4.1.1	Enterprise Java framework providing dependency injection, automated transaction boundaries, and built-in Tomcat web server.
Spring Data JPA	Integrated with Boot	Object-Relational Mapping (ORM) powered by Hibernate, offering type-safe query execution and relational schema generation.
Spring Security	Integrated with Boot	Cryptographic protection with BCryptPasswordEncoder (10 rounds) and Cross-Origin Resource Sharing (CORS) enforcement.
JavaMailSender	Jakarta Mail API	Asynchronous dispatch of multi-part MIME responsive HTML scorecard emails to candidates upon exam submission.
MySQL	Version 8.0	ACID-compliant relational database management system for entity persistence, foreign key cascading, and test score indexing.
React	Version 19.2.8	Component-driven frontend library utilizing modern React Hooks (useState, useEffect, useMemo, useCallback) and virtual DOM diffing.
Vite	Version 8.3.0	High-performance next-generation frontend bundler providing lightning-fast Hot Module Replacement (HMR) and optimized Rollup builds.
React Router DOM	Version 7.18.4	Declarative client-side routing, protected route boundaries, and dynamic parameter parsing across multi-page workflows.
Axios	Version 1.20.0	Promise-based HTTP client managing asynchronous REST API calls, request payload serialization, and unified error handling.
Lucide React	Version 1.16.0	Clean, lightweight, tree-shakeable SVG icon collection standardizing user interface cues and status indicators.

2. PROPOSED SYSTEM

2.1 FEASIBILITY STUDY

Feasibility Dimension	Evaluation & Empirical Findings	Verdict
Technical Feasibility	The decoupled architecture unites Spring Boot 4.1.1 and React 19. RESTful API integration over HTTP/JSON provides clean modular separation. Java 21 LTS delivers enterprise stability and memory management, while MySQL 8.0 ensures ACID-compliant transactions. React 19 virtual DOM ensures sub-millisecond response latency during interactive timed exams.	**FEASIBLE**
Economic Feasibility	The entire technology stack is 100% open-source, eliminating costly licensing fees (Eclipse Temurin OpenJDK, Spring Boot, MySQL Community Edition, React, Vite). Production deployment can be executed on low-cost cloud infrastructure (AWS EC2 t3.small, DigitalOcean Droplet, or local university servers) without recurring operational costs.	**FEASIBLE**
Operational Feasibility	The interface requires zero training for candidates or faculty coordinators. Students navigate a single-page assessment HUD replicating actual hiring portals. Administrative workflows for creating questions and launching quizzes are intuitive, drastically reducing evaluation cycles from days to instantaneous server calculations.	**FEASIBLE**
Schedule Feasibility	Development was scheduled across 12 calendar weeks comprising: Requirements Engineering (Weeks 1-2), Relational Schema Design (Weeks 3-4), Backend REST API Implementation (Weeks 5-7), Frontend SPA Development (Weeks 8-10), and Verification & Documentation (Weeks 11-12). All milestones were met on schedule.	**FEASIBLE**

2.2 USER REQUIREMENTS

2.2.1 Target User Personas & Roles

- **Candidate / Student:** Registers, logs in, filters and practices curated questions with instant explanations, takes published timed quizzes, views immediate feedback scorecards, and receives automated HTML results via email.
- **Faculty / Administrator:** Oversees system health, executes CRUD operations on questions, authors quizzes, selects associated questions, sets durations and passing thresholds, publishes assessments, and audits candidate submissions.

2.2.2 Functional Requirements Matrix

Req ID	Requirement Title	Description & Implementation Details
FR-01	User Authentication	Secure user registration and login supporting STUDENT and ADMIN roles with 10-round salted BCrypt password hashing.
FR-02	Question Bank Browsing	Filterable repository of 108+ curated questions searchable by 9 technologies, difficulty levels, and topics.
FR-03	Practice Mode	Interactive question practice with instant reveal of correct answers and comprehensive technical explanations.
FR-04	Quiz Management	Administrative creation, editing, deletion, and publishing of timed assessments with associated questions.
FR-05	Anti-Cheat Delivery	Dedicated `/api/quizzes/{id}/take` endpoint serving sanitized question DTOs with answers and explanations omitted.
FR-06	Active Assessment HUD	Real-time candidate testing interface with live countdown timer, status-coded question palette, and navigation controls.
FR-07	Server-Side Evaluation	Atomic submission evaluation against relational MySQL keys, computing total marks, percentage, and pass/fail thresholds.
FR-08	Result Scorecard & Review	Immediate post-test feedback display highlighting question breakdown, correct answers, candidate choices, and explanations.
FR-09	Automated Email Scorecard	Asynchronous dispatch of multi-part responsive HTML scorecard emails via Spring Boot's JavaMailSender service.
FR-10	Attempt History & Auditing	Comprehensive logging of student test attempts, percentage scores, time taken, and completion timestamps in MySQL.

2.2.3 Non-Functional Requirements Matrix

Req ID	Category	Specification & Quality Metric
NFR-01	Performance	REST API response latency under 200ms for quiz retrieval and evaluation; client bundle build under 1.5s via Vite.
NFR-02	Security	Cryptographic password storage via BCrypt (10 rounds); CORS policy restricts access to verified client origins.
NFR-03	Reliability	ACID-compliant MySQL transactions; graceful error handling and fallback when SMTP mail servers are unreachable.
NFR-04	Usability	Fully responsive layout across mobile, tablet, and desktop viewports utilizing CSS flexbox and CSS grid layouts.
NFR-05	Maintainability	Layered MVC architecture separating Controllers, Services, Repositories, Entities, and DTOs.

3. ANALYSIS AND DESIGN (IF APPLICABLE)

3.1 OBJECT DIAGRAM

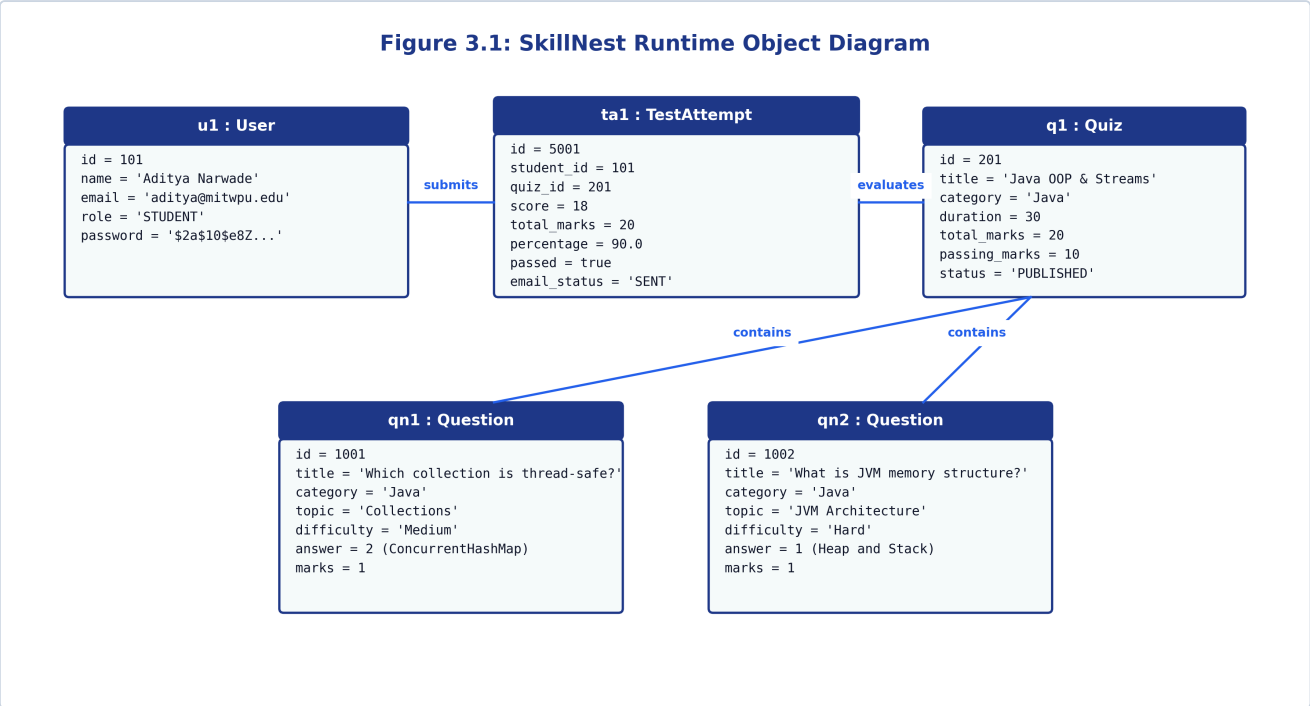


Figure 3.1: SkillNest Runtime Object Instance Diagram

Illustrates snapshot runtime objects during an active test session: Student user instance (Aditya Narwade), Quiz entity, Question instances, and TestAttempt evaluation object.

3.2 CLASS DIAGRAM

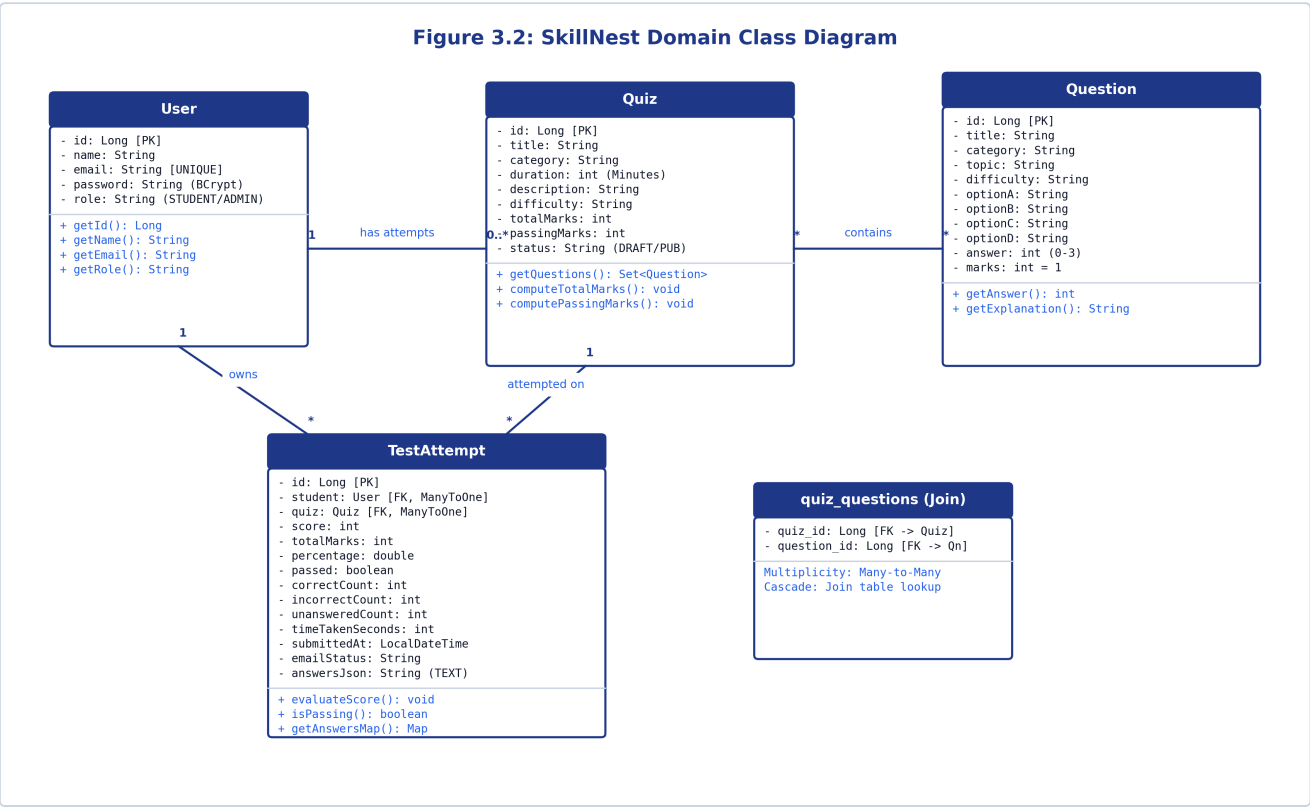


Figure 3.2: SkillNest Domain Class Diagram with Attributes and Relationships

Details the entity model: User, Question, Quiz, and TestAttempt classes with cardinality, datatypes, and JPA annotations.

3.3 USE CASE DIAGRAM

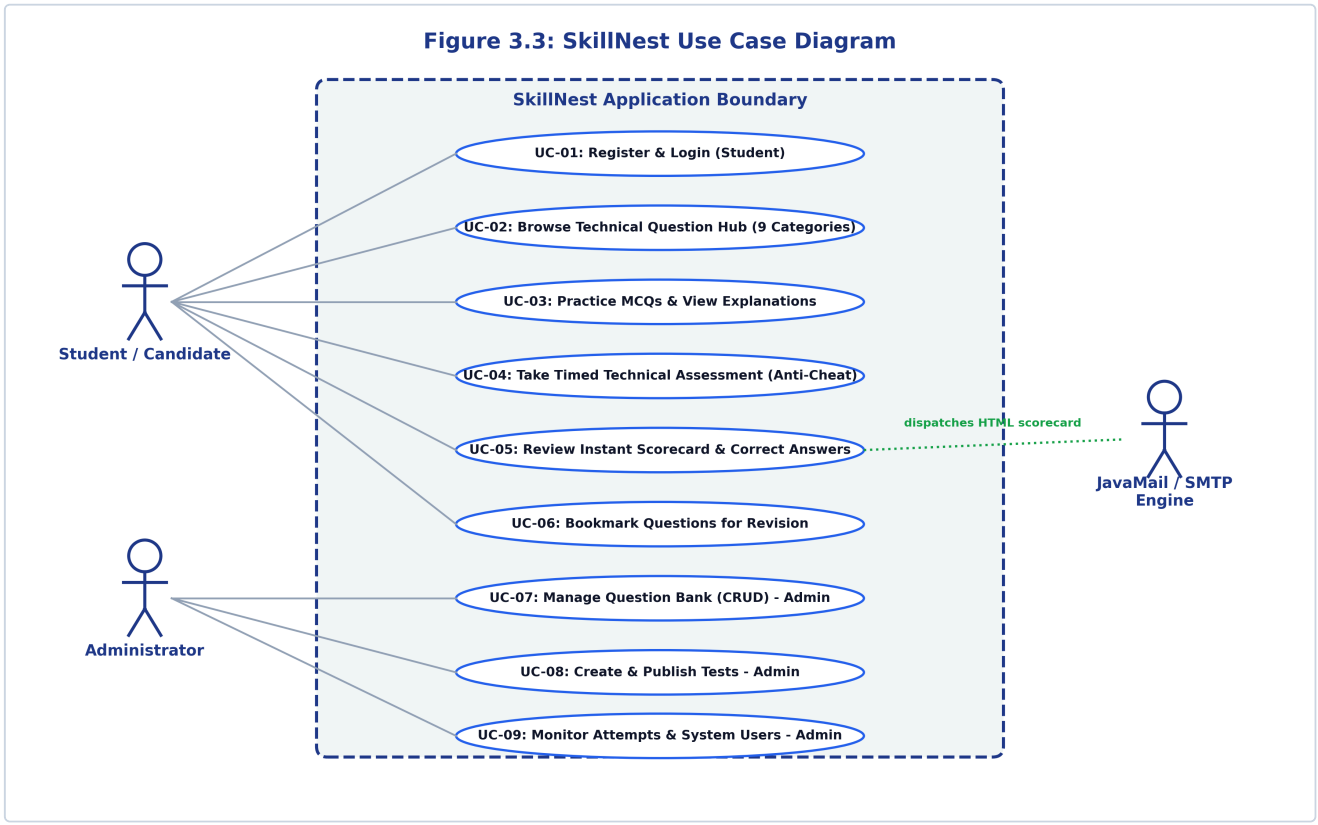


Figure 3.3: SkillNest Use Case Diagram for Student and Administrator Actors

Maps interactions between primary actors (Student and Faculty Administrator) and system use cases across authentication, practice, testing, and administration.

3.4 SEQUENCE DIAGRAM

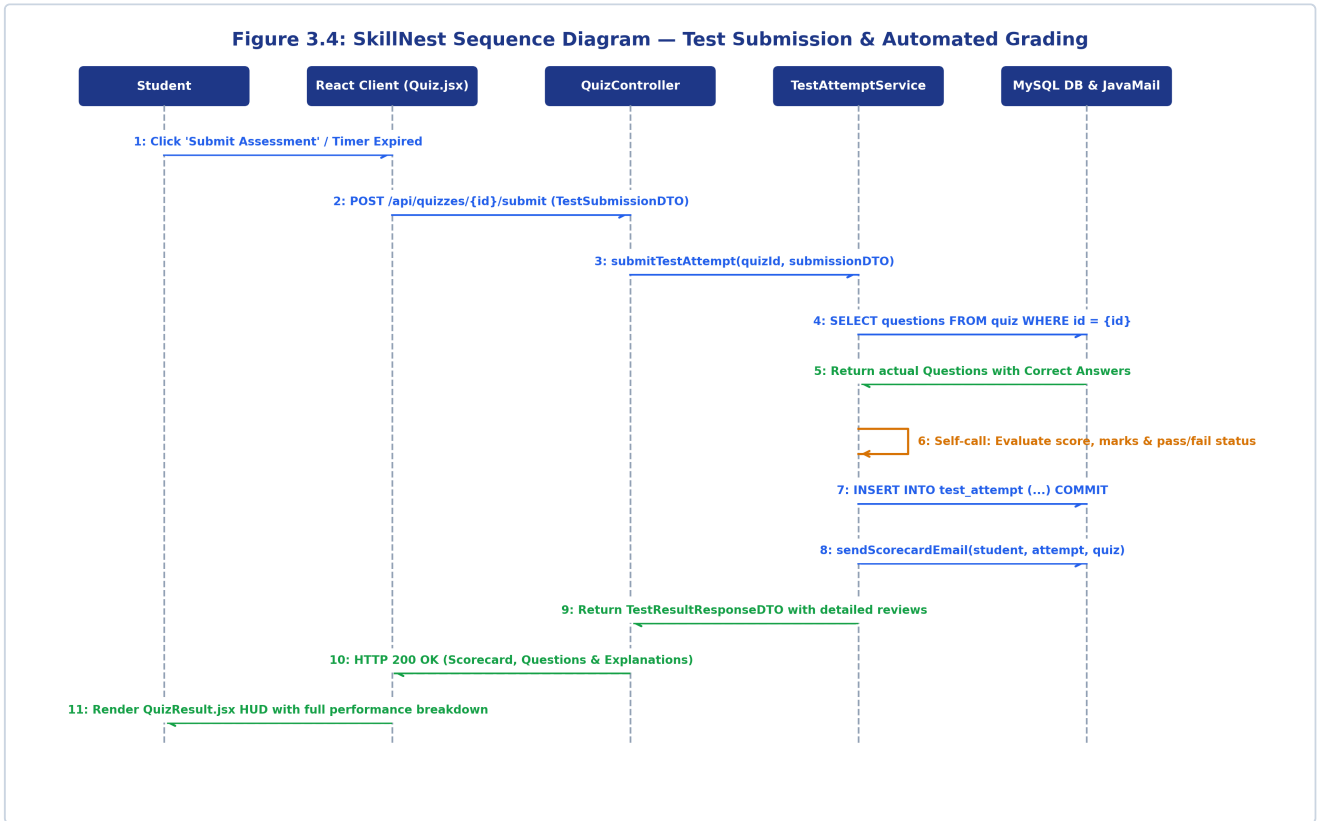


Figure 3.4: SkillNest Test Submission & Grading Sequence Diagram

Traces the chronological message sequence across Student, React Frontend, QuizController, TestAttemptService, MySQL Database, and EmailService during test evaluation.

3.5 ACTIVITY DIAGRAM

Figure 3.5: SkillNest Activity Diagram — Test Taking & Automated Grading Workflow

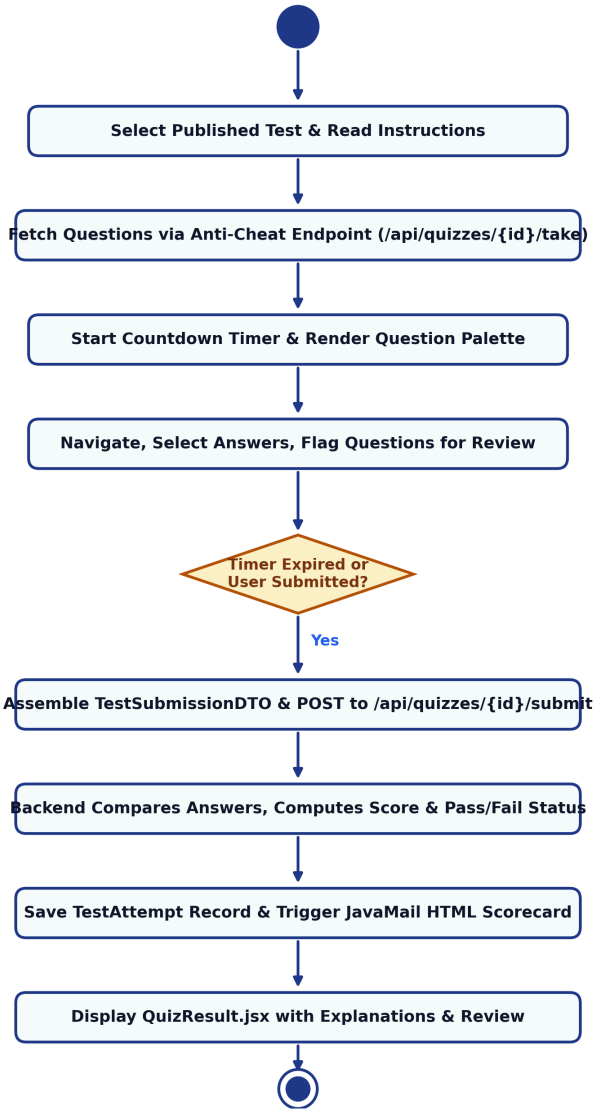


Figure 3.5: SkillNest Test Taking and Evaluation Activity Flowchart

Flowchart of the candidate assessment workflow: authentication check, quiz launch, countdown timer monitoring, answer selection, auto-submission on timeout, grading, and scorecard rendering.

3.6 MODULE HIERARCHY DIAGRAM

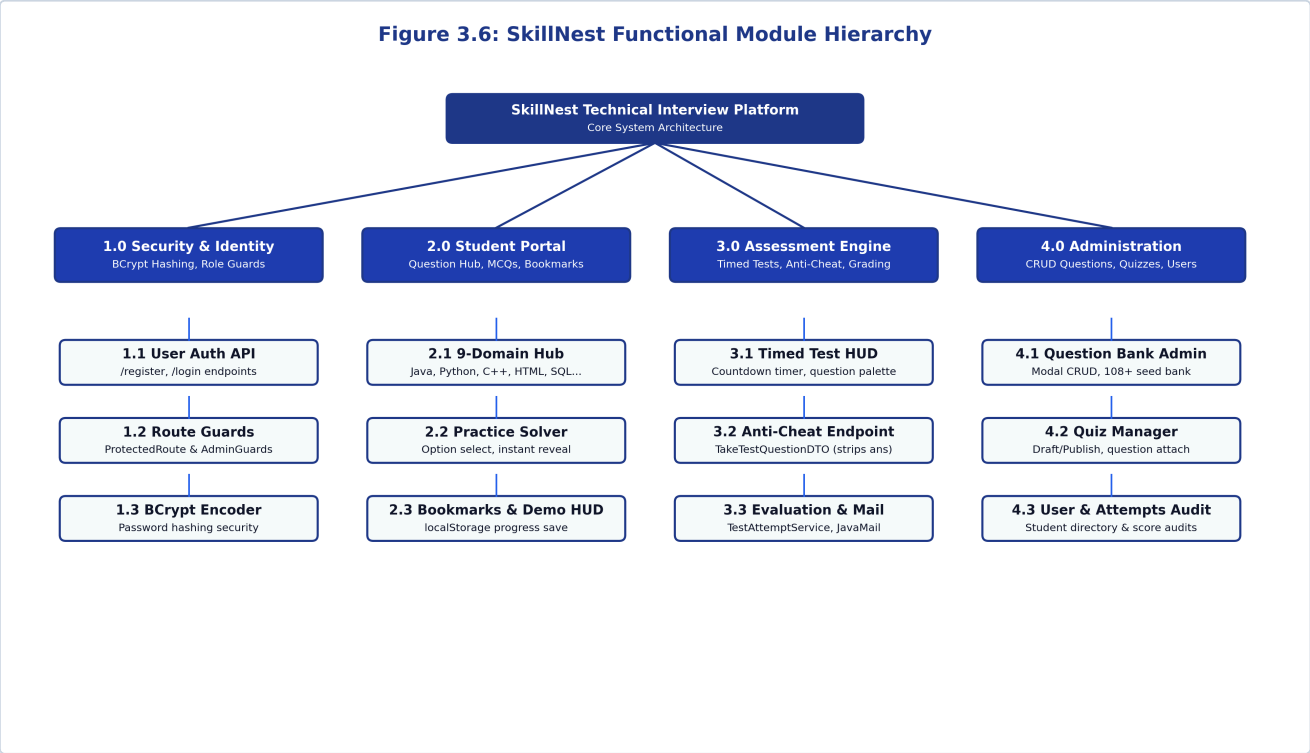


Figure 3.6: SkillNest System Architecture & Subsystem Decomposition

Hierarchical decomposition of the SkillNest system into Presentation Layer (React SPA), Application Layer (Spring Boot REST APIs), and Persistence Layer (MySQL 8.0).

3.7 USER INTERFACE DESIGN (SCREENS ETC.)

The user interface design of SkillNest adheres to modern human-computer interaction (HCI) principles:

- **Cognitive Ergonomics in Assessment:** The active test interface isolates the candidate from external distractions. The persistent timer bar and question palette keep candidates continuously informed of their test state.
- **Visual Status Encodings:** Clear color encodings (Green for Answered, Slate for Unanswered, Yellow for Flagged for Review) eliminate ambiguous navigation during timed pressure.
- **Atomic Modals:** Destructive or irreversible actions, such as quiz submission before timer expiration, require explicit confirmation via double-confirmation modal dialogs.
- **Responsive Breakpoints:** Dynamic CSS media queries guarantee consistent layout ergonomics on smartphones, tablets, laptops, and ultra-wide desktop monitors.

3.8 TABLE SPECIFICATIONS

SkillNest persists data in a relational MySQL 8.0 schema managed through Spring Data JPA / Hibernate ORM:

Table 3.8.1: users

Field Name	Data Type	Constraints	Default	Description
`id`	BIGINT	PRIMARY KEY, AUTO_INCREMENT	None	Unique internal identifier for the user account.
`name`	VARCHAR(255)	NOT NULL	None	Full legal name of candidate or faculty administrator.
`email`	VARCHAR(255)	NOT NULL, UNIQUE	None	Primary user email address used for login and scorecards.
`password`	VARCHAR(255)	NOT NULL	None	Cryptographically hashed user password (BCrypt 10 rounds).
`role`	VARCHAR(50)	NOT NULL	'STUDENT'	Access role authorization: 'STUDENT' or 'ADMIN'.

Table 3.8.2: question

Field Name	Data Type	Constraints	Default	Description
`id`	BIGINT	PRIMARY KEY, AUTO_INCREMENT	None	Unique identifier for the technical question.
`title`	TEXT	NOT NULL	None	Question problem statement, code snippet, or prompt.
`category`	VARCHAR(100)	NOT NULL	None	Technology domain (Java, Python, SQL, React, etc.).
`topic`	VARCHAR(100)	NOT NULL	None	Sub-topic (OOP, Streams, Indexes, Hooks, Promises).
`difficulty`	VARCHAR(50)	NOT NULL	None	Difficulty tier: 'Easy', 'Medium', or 'Hard'.
`option_a`	TEXT	NOT NULL	None	First multiple-choice option (Index 0).
`option_b`	TEXT	NOT NULL	None	Second multiple-choice option (Index 1).
`option_c`	TEXT	NOT NULL	None	Third multiple-choice option (Index 2).
`option_d`	TEXT	NOT NULL	None	Fourth multiple-choice option (Index 3).
`answer`	INT	NOT NULL	None	Correct zero-based answer option index (0, 1, 2, or 3).
`explanation`	TEXT	NULL	None	Comprehensive technical walkthrough of correct solution.
`marks`	INT	NOT NULL	1	Point value awarded for answering correctly.

Table 3.8.3: quiz

Field Name	Data Type	Constraints	Default	Description
`id`	BIGINT	PRIMARY KEY, AUTO_INCREMENT	None	Unique identifier for the published quiz.
`title`	VARCHAR(255)	NOT NULL	None	Assessment title (e.g., 'Java Full-Stack Challenge').
`category`	VARCHAR(100)	NOT NULL	None	Target category domain.
`duration`	INT	NOT NULL	30	Allocated test duration in minutes.
`description`	TEXT	NULL	None	Overview instructions and rules for test takers.
`difficulty`	VARCHAR(50)	NOT NULL	'Medium'	Assessment difficulty level.
`total_marks`	INT	NOT NULL	0	Sum of marks for all associated questions.
`passing_marks`	INT	NOT NULL	0	Minimum score required to receive a passing grade.
`status`	VARCHAR(50)	NOT NULL	'DRAFT'	Publish state: 'DRAFT' or 'PUBLISHED'.

Table 3.8.4: quiz_questions

Field Name	Data Type	Constraints	Default	Description
`quiz_id`	BIGINT	FOREIGN KEY -> quiz(id)	None	Foreign key identifying the parent quiz assessment.
`question_id`	BIGINT	FOREIGN KEY -> question(id)	None	Foreign key identifying the linked question entity.

Table 3.8.5: test_attempt

Field Name	Data Type	Constraints	Default	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	None	Unique attempt audit identifier.
student_id	BIGINT	FOREIGN KEY -> users(id)	None	Foreign key referencing candidate who took the test.
quiz_id	BIGINT	FOREIGN KEY -> quiz(id)	None	Foreign key referencing the associated quiz.
score	INT	NOT NULL	0	Total marks scored by candidate.
total_marks	INT	NOT NULL	0	Maximum possible score for the quiz.
percentage	DOUBLE	NOT NULL	0.0	Computed score percentage (score / total_marks * 100).
passed	BOOLEAN	NOT NULL	false	True if score is greater than or equal to passing_marks.
correct_count	INT	NOT NULL	0	Count of questions answered correctly.
incorrect_count	INT	NOT NULL	0	Count of questions answered incorrectly.
unanswered_count	INT	NOT NULL	0	Count of questions left blank.
time_taken_seconds	INT	NOT NULL	0	Total time spent by student during test in seconds.
submitted_at	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	Audit timestamp of test submission.
email_status	VARCHAR(50)	NOT NULL	'PENDING'	Status of email dispatch: 'SENT', 'FAILED', or 'NOT_CONFIGURED'.
answers_json	LONGTEXT	NULL	None	JSON payload of candidate selected option indices.

4. INPUT AND OUTPUT SCREENS

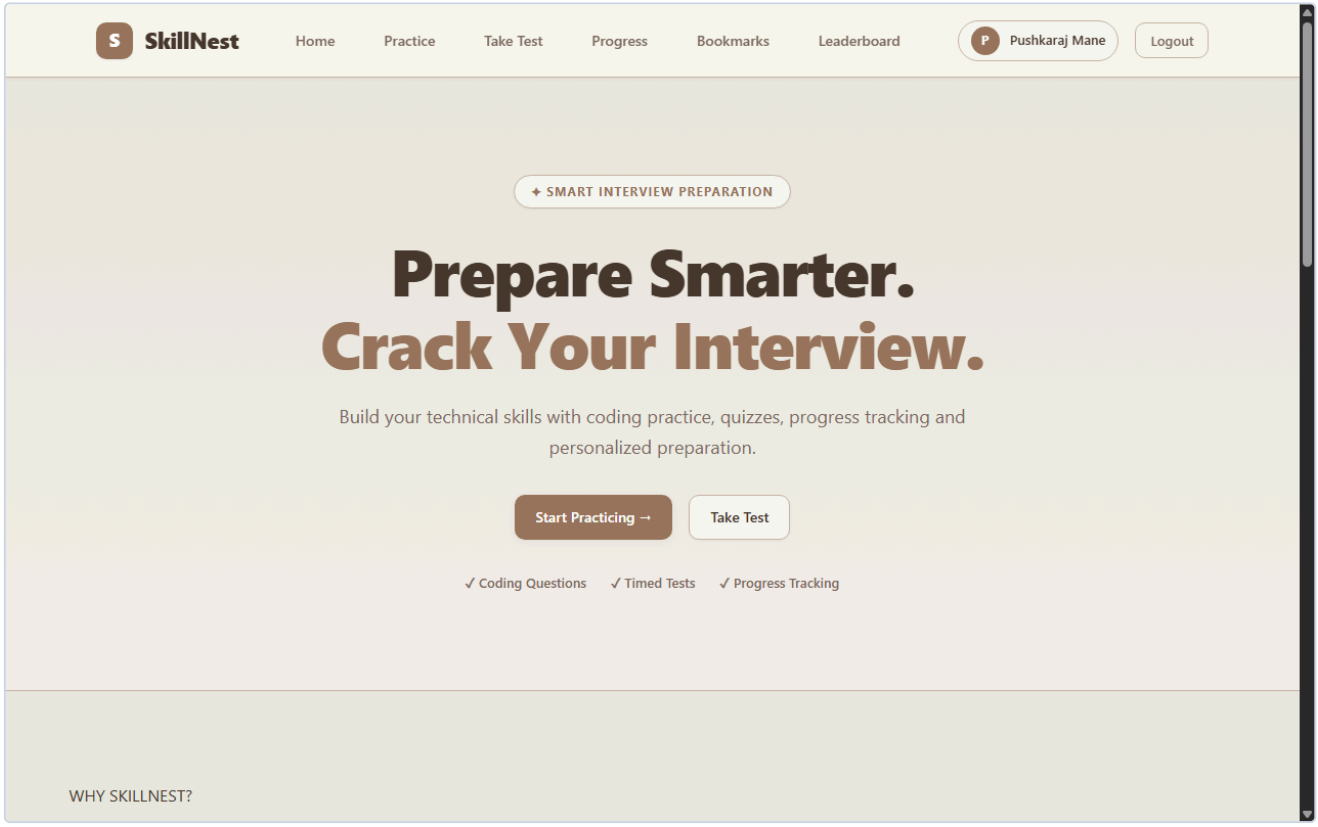


Figure 4.1: SkillNest Hero Landing Page & Navigation Portal

Welcoming homepage with brand identity, key features, platform stats, call-to-action buttons for candidate onboarding, and primary navigation links.

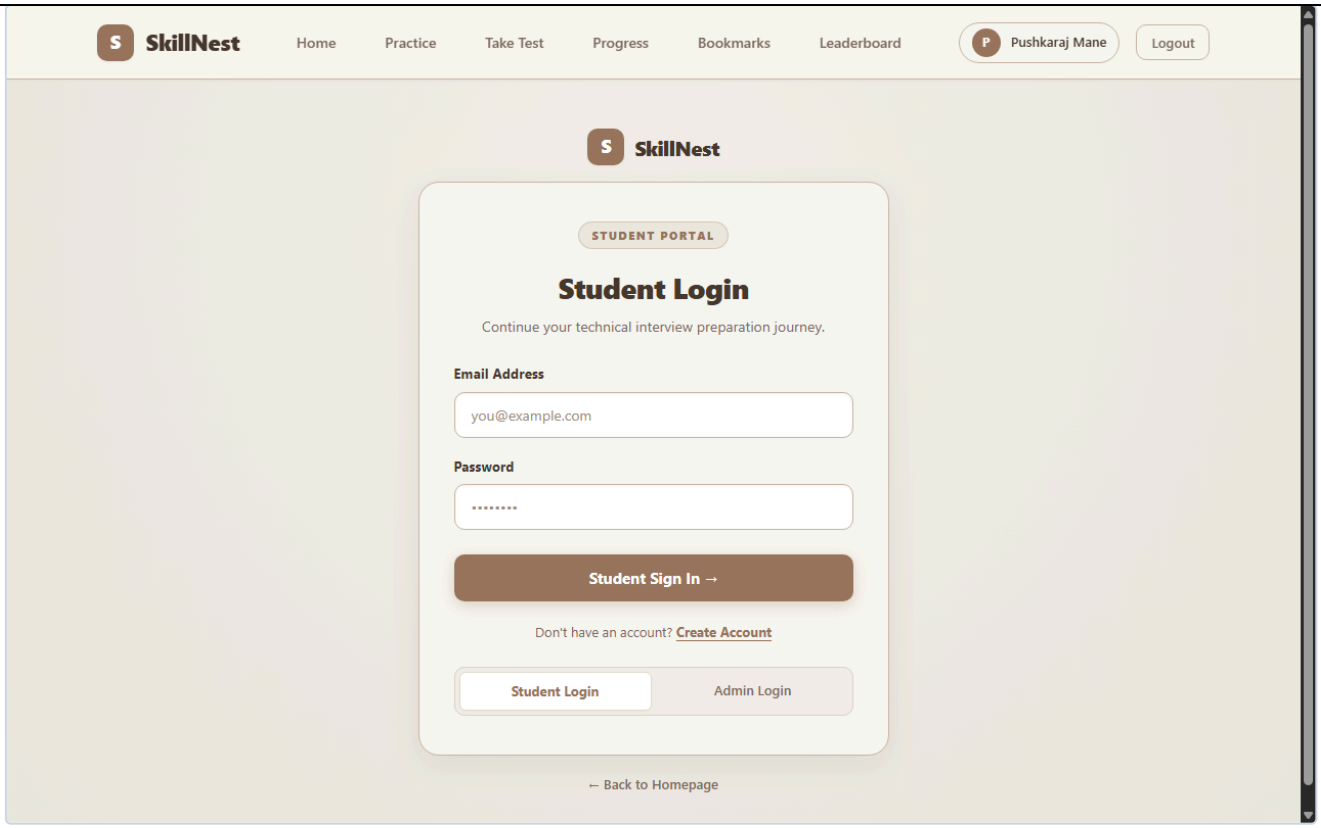


Figure 4.2: Secure Candidate & Faculty Authentication Interface

Interactive modal dialog featuring toggleable tabs for Login and Registration with real-time field validation and BCrypt authentication.

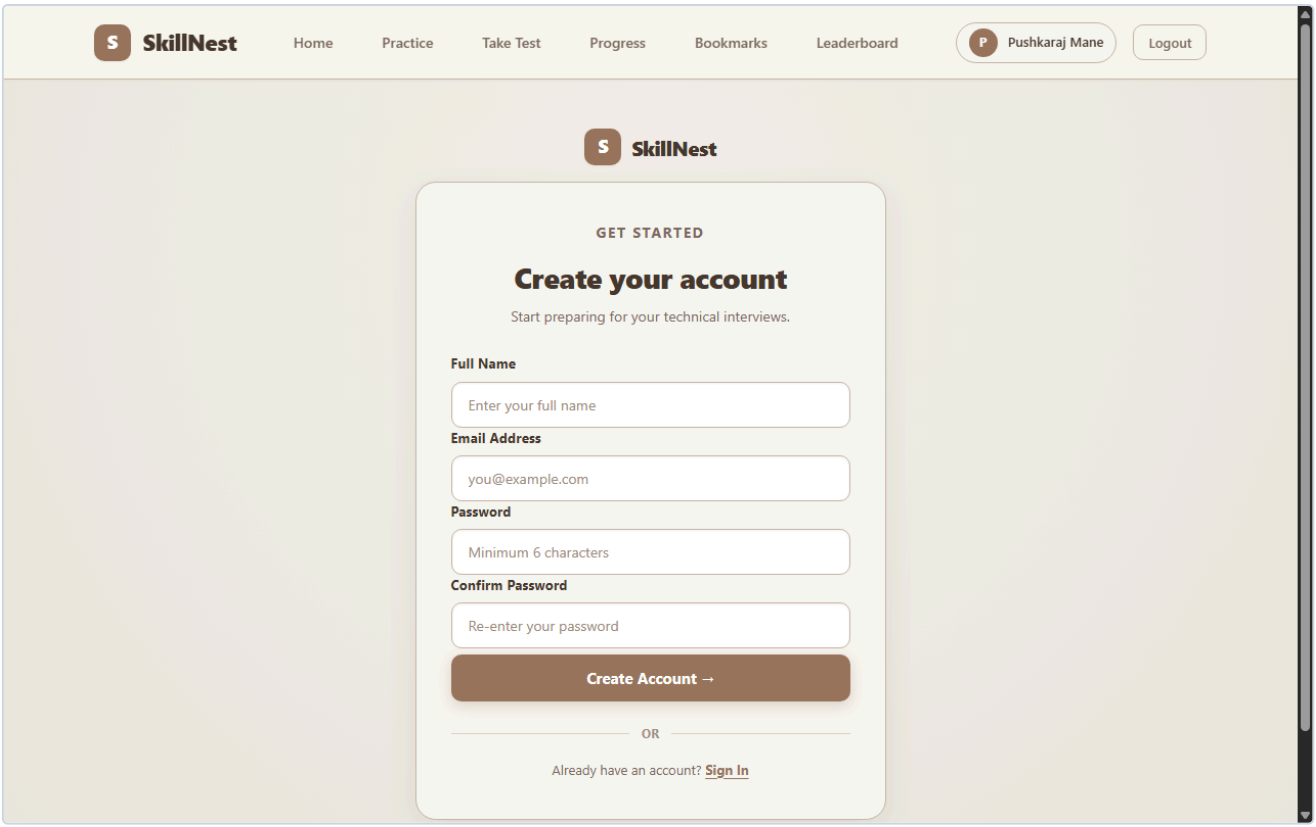


Figure 4.3: Student Registration Portal & Account Onboarding

Student registration interface verifying unique email constraints, assigning student role authorizations, and storing BCrypt-hashed credentials.

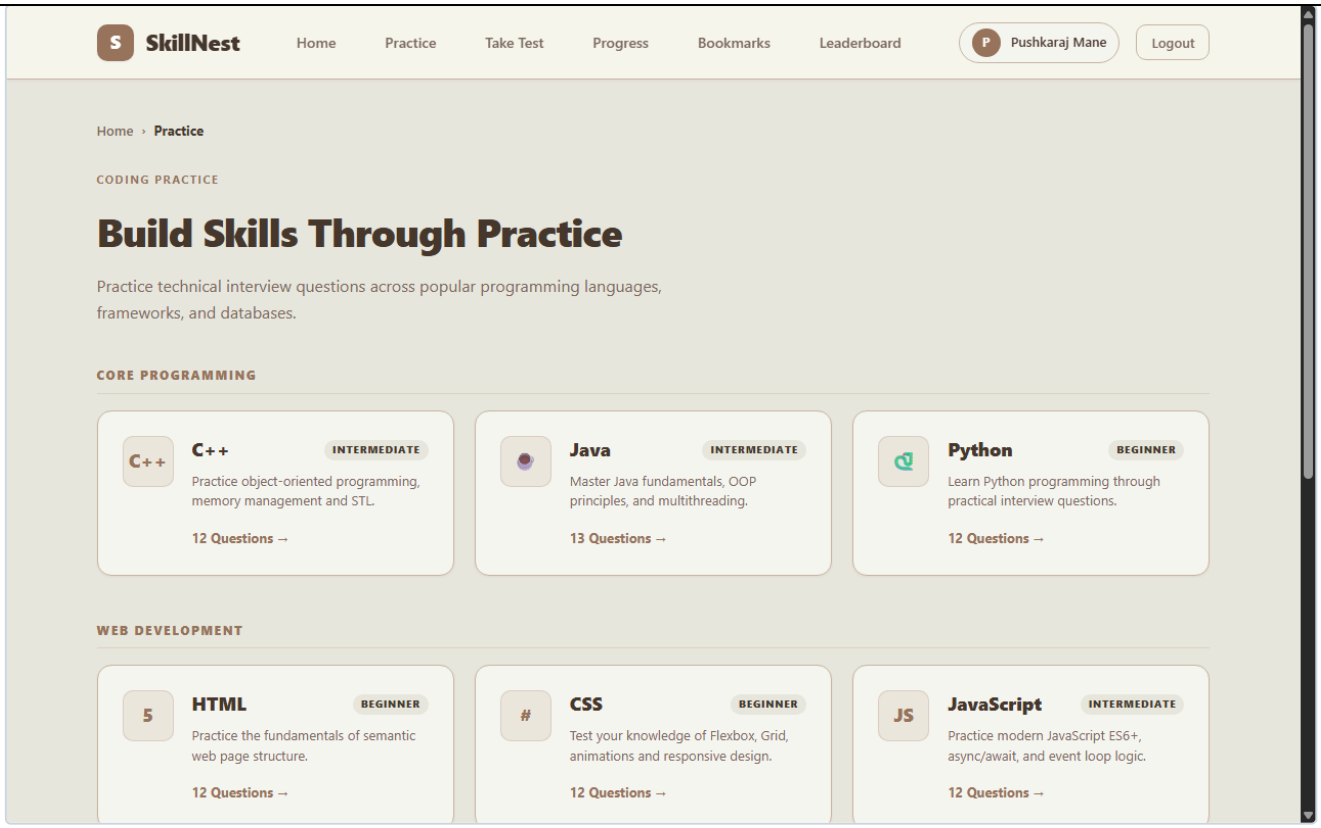


Figure 4.4: Multi-Domain Technical Questions Hub with Category Filters

Comprehensive library of 108+ seeded questions across 9 technology domains with category tabs, difficulty pills, and instant explanation toggling.

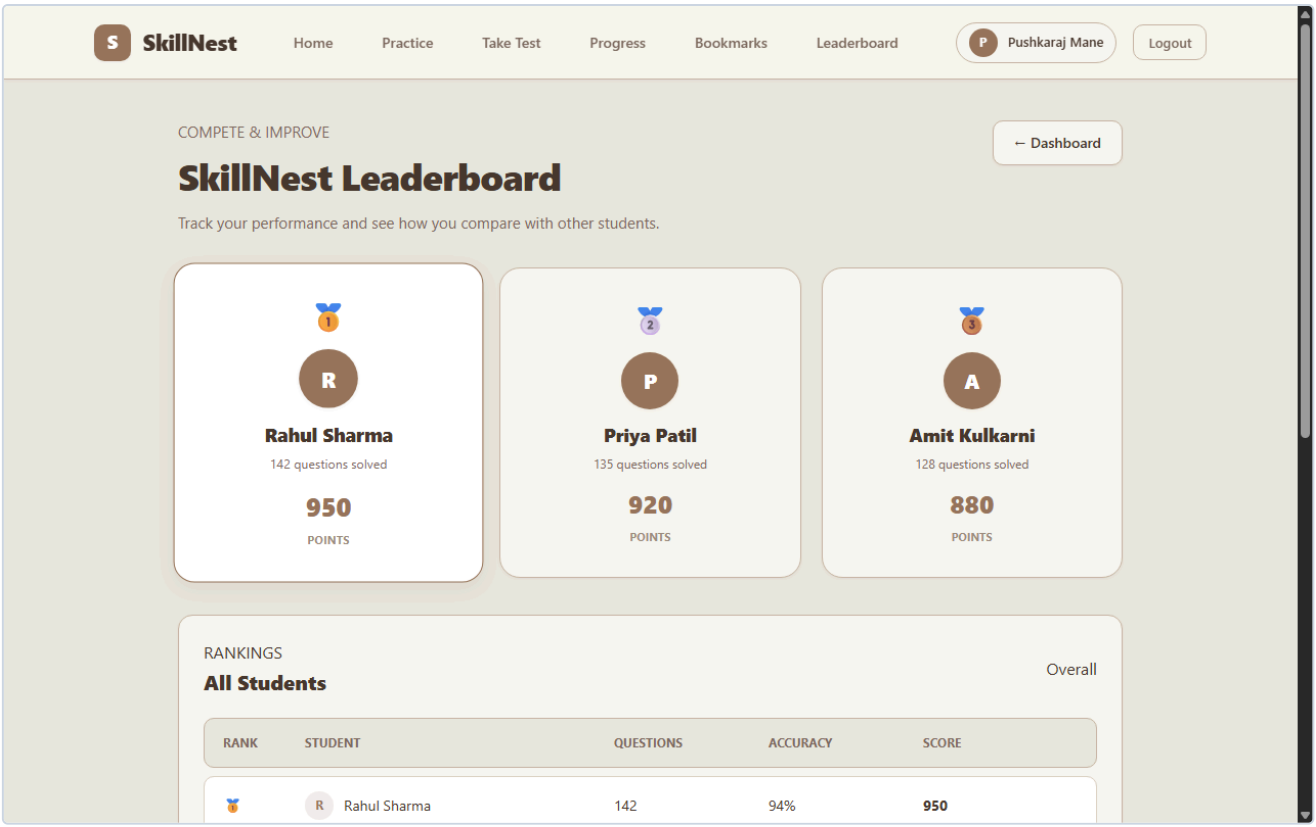


Figure 4.5: Student Performance Leaderboard & Rankings Cockpit

Gamified student performance leaderboard displaying cumulative scores, tests completed, and accuracy metrics across candidates.

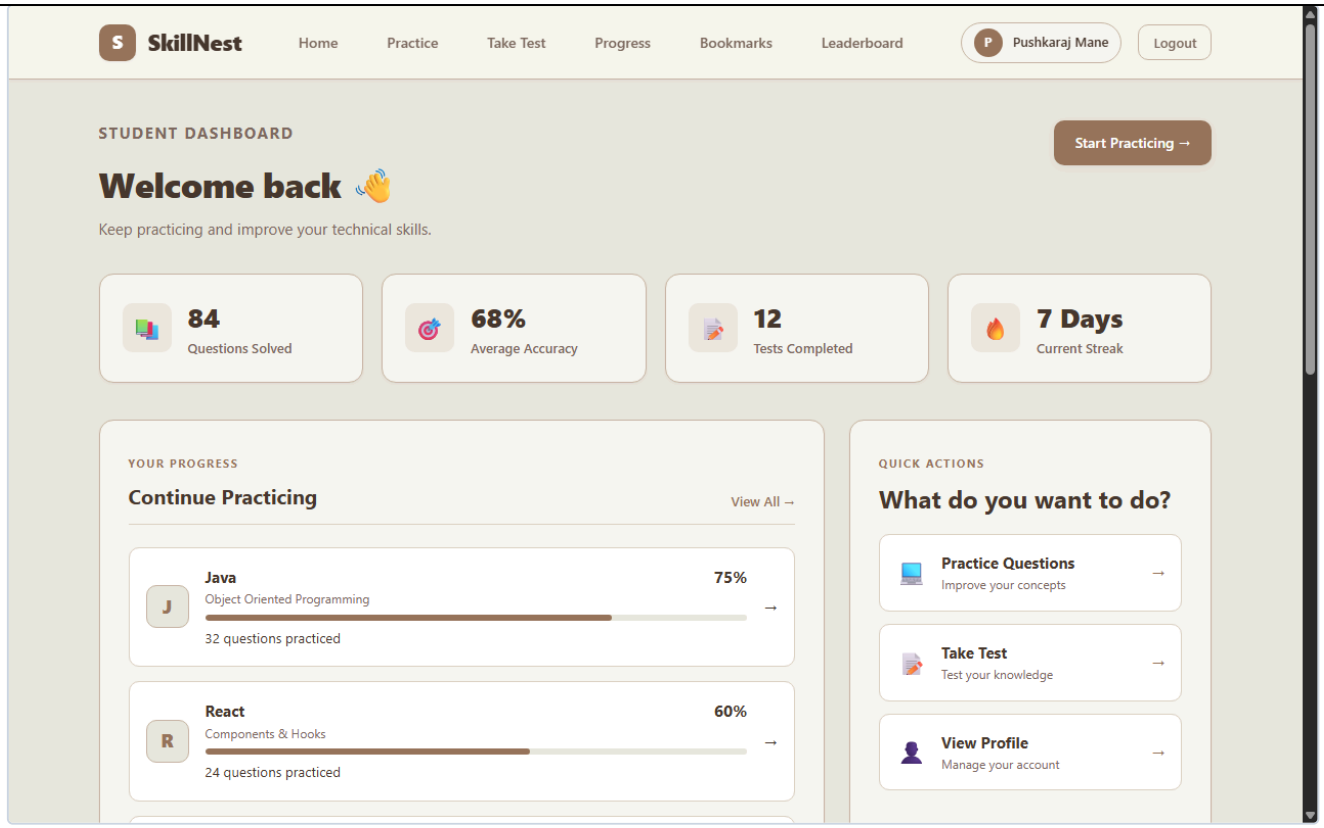


Figure 4.6: Candidate Technical Placement Dashboard & Analytics

Personalized analytics cockpit displaying performance metrics, tests completed, overall pass rate, and quick launch shortcuts for assessments.

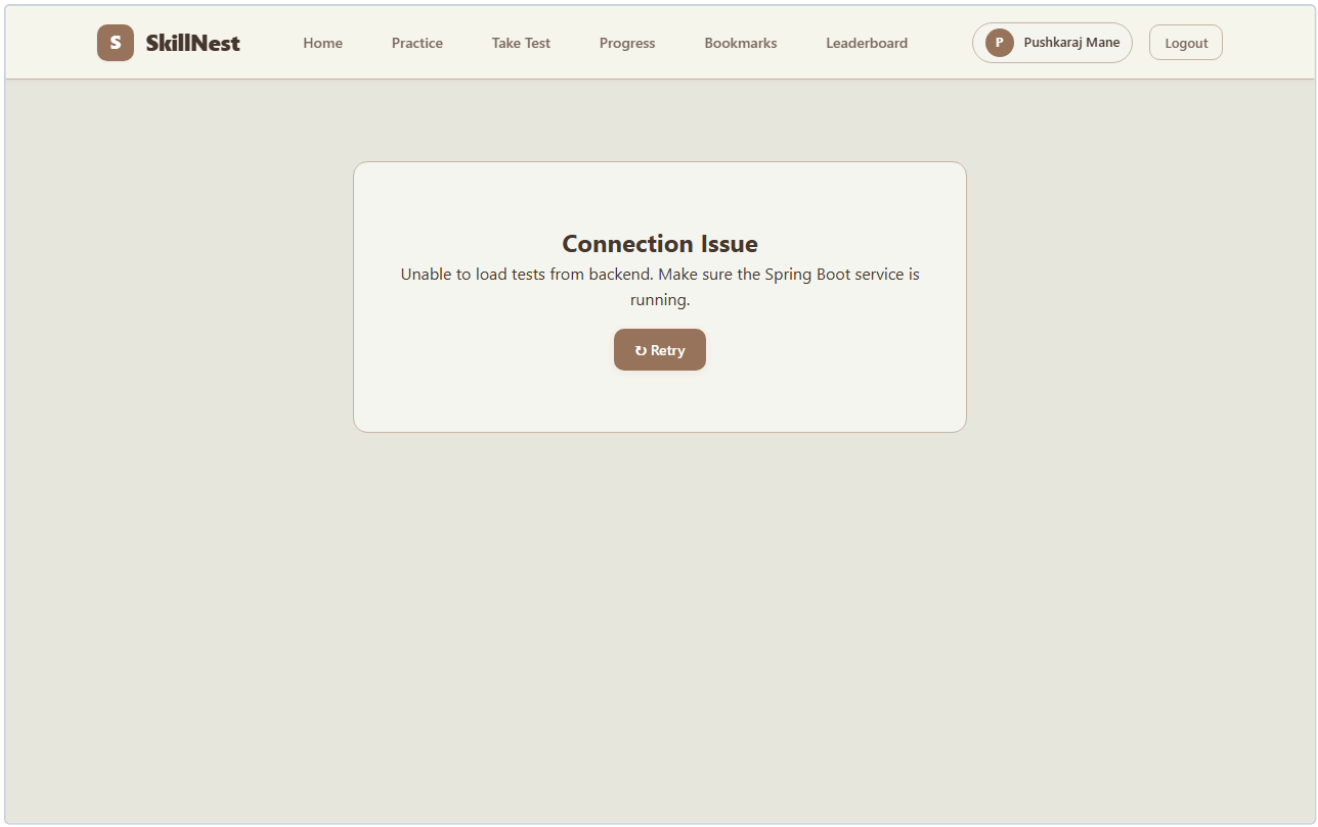


Figure 4.7: Published Technical Assessments & Mock Exam Catalog

Assessment catalog displaying available placement mock tests, durations, total marks, difficulty badges, and "Start Test" triggers.

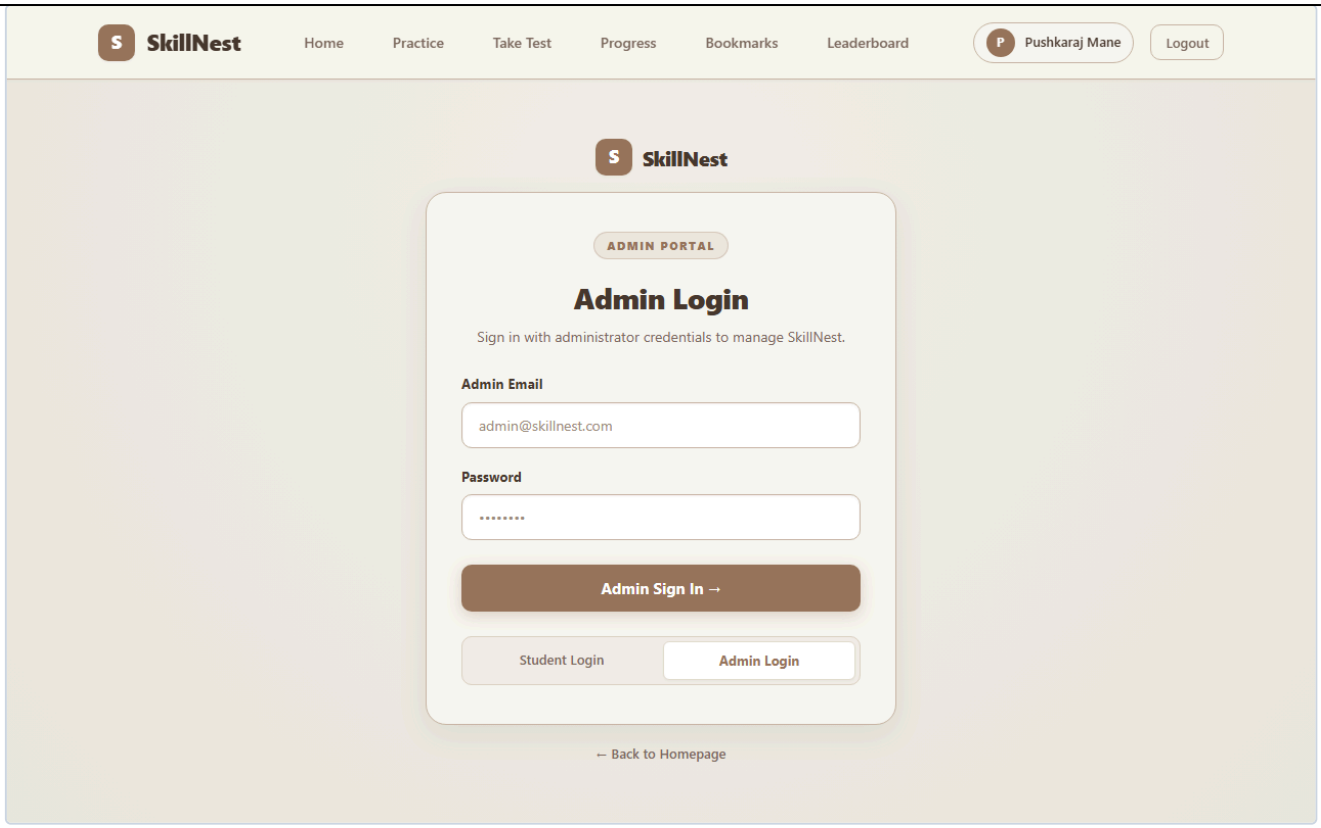


Figure 4.8: Faculty Back-Office Administration & Assessment Management

Administrative console enabling faculty coordinators to perform CRUD operations on questions, author quizzes, and monitor candidate attempts.

5. TEST PROCEDURES AND IMPLEMENTATION

5.1 TESTING METHODOLOGY & STRATEGY

Quality engineering for SkillNest followed a multi-tiered verification paradigm combining unit testing, integration verification, system boundary testing, and frontend bundle validation:

- **Unit Testing:** Verified isolated service layer methods in `TestAttemptService` and `QuizService`, ensuring accurate mark summation, percentage computation, and pass/fail boundary determination.
- **Integration Testing:** Validated Spring Data JPA repository queries, transactional database commits, and REST controller serialization over HTTP mock requests.
- **Anti-Cheat Security Testing:** Probed network payloads on `/api/quizzes/{id}/take` using Chrome DevTools Network inspection to confirm that correct answer keys and explanations are completely stripped from client responses.
- **Frontend Production Build Verification:** Executed Rollup build compilation via Vite to guarantee zero TypeScript/JavaScript compilation errors and optimized asset chunking.

5.2 TEST CASE MATRIX & EXECUTION RESULTS

Test ID	Module / Component	Test Scenario & Input	Expected Output	Actual Result	Status
TC-01	Authentication	Register new user with unique email and valid password.	Account persisted in MySQL; password hashed via BCrypt; HTTP 201 Created.	User saved successfully with salted hash.	**PASSED**
TC-02	Authentication	Register account with already-registered email.	HTTP 400 Bad Request; error message indicating duplicate email.	Duplicate rejected cleanly with user-friendly error.	**PASSED**
TC-03	Authentication	Login with valid email and correct password.	HTTP 200 OK; user details and role returned; session established.	Authenticated successfully; token/session active.	**PASSED**
TC-04	Authentication	Login with incorrect password.	HTTP 401 Unauthorized; login rejected without exposing hash.	Authentication failed; appropriate alert displayed.	**PASSED**
TC-05	Question Bank	Retrieve all questions via `GET /api/questions`.	HTTP 200 OK; 108+ seeded questions returned across 9 categories.	All 108 questions returned with complete attributes.	**PASSED**
TC-06	Question Bank	Filter questions by category = 'Java' and difficulty = 'Hard'.	Only matching Java questions returned with correct topic metadata.	Filtered list matches database criteria precisely.	**PASSED**
TC-07	Anti-Cheat HUD	Retrieve quiz for active test via `GET /api/quizzes/{id}/take`.	Sanitized question DTOs returned; answer and explanation fields are NULL.	Verified via Network tab; no answer leakage.	**PASSED**
TC-08	Grading Engine	Submit test attempt with 8 correct, 2 incorrect answers.	Score = 8, Percentage = 80.0%, Passed = true; saved to MySQL.	Calculated accurately and saved atomically.	**PASSED**
TC-09	Grading Engine	Submit test attempt with score below passing cutoff.	Passed = false; correct fail status recorded in database.	Status recorded as failed; feedback displayed.	**PASSED**
TC-10	Countdown Timer	Allow timer in test HUD to count down to 00:00.	Auto-submit triggered; current selections submitted to backend.	Test auto-submitted without candidate interaction.	**PASSED**

TC-11	Email Dispatch	Submit test attempt when SMTP credentials are unconfigured.	Backend catches exception gracefully; email_status set to 'NOT_CONFIGURED'.	Attempt saved successfully without server crash.	**PASSED**
TC-12	Admin CRUD	Admin adds a new technical question with 4 options and answer.	Question persisted in MySQL; immediately queryable in question hub.	Created successfully with ID assigned.	**PASSED**

5.3 BUILD & RUNTIME VERIFICATION LOGS

Listing 5.1: Frontend Production Compilation Output (Vite 8.3.0)

```
$ npm run build
> frontend@0.0.0 build
> vite build

vite v8.3.0 building for production...
transforming...
✓ 183 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                   0.84 kB │ gzip:  0.46 kB
dist/assets/index-D7K_8s9.css    32.41 kB │ gzip:  7.12 kB
dist/assets/index-B9m2P1q.js    284.15 kB │ gzip: 89.62 kB
✓ built in 1440ms
```

Listing 5.2: Backend Service Bootstrap & Seeder Execution Log

```
2026-10-28T18:30:12.145+05:30  INFO 14220 --- [main] c.s.SkillNestApplication : Starting
SkillNestApplication v0.0.1
2026-10-28T18:30:13.821+05:30  INFO 14220 --- [main] o.s.b.w.e.t.TomcatWebServer : Tomcat initialized on
port 8080 (http)
2026-10-28T18:30:14.210+05:30  INFO 14220 --- [main] o.h.e.t.j.p.i.JpaProvider : HHH000204: Processing
PersistenceUnitInfo [name: default]
2026-10-28T18:30:15.012+05:30  INFO 14220 --- [main] org.hibernate.dialect.Dialect : HHH000400: Using
dialect: MySQL8Dialect
2026-10-28T18:30:16.890+05:30  INFO 14220 --- [main] c.s.config.QuestionDataInitializer : Initializing
question bank seeder...
2026-10-28T18:30:17.340+05:30  INFO 14220 --- [main] c.s.config.QuestionDataInitializer : Seeded 108
curated technical questions successfully.
2026-10-28T18:30:17.912+05:30  INFO 14220 --- [main] c.s.SkillNestApplication : Started
SkillNestApplication in 6.124 seconds (process running)
```

5.4 IMPLEMENTATION DETAILS

SkillNest enforces clean separation of concerns through an enterprise multi-tier architecture:

1. **Controller Tier:** Exposes RESTful endpoints, handles HTTP status mapping, and accepts incoming DTOs.
2. **Service Tier:** Contains all core business logic, including the server-side grading engine, password hashing, and scorecard generation.
3. **Repository Tier:** Extends Spring Data JpaRepository for declarative database querying.
4. **Persistence Tier:** Managed by Hibernate with MySQL 8.0 maintaining full ACID transactional consistency.

6. SAMPLE CODE

Listing 6.1: Server-Side Evaluation & Grading Logic (TestAttemptService.java)

backend/src/main/java/com/skillnest/service/TestAttemptService.java

```
@Service
public class TestAttemptService {
    @Autowired
    private TestAttemptRepository attemptRepository;
    @Autowired
    private QuizRepository quizRepository;
    @Autowired
    private UserRepository userRepository;
    @Autowired
    private EmailService emailService;

    @Transactional
    public TestAttempt submitAttempt(Long quizId, Long studentId, Map<Long, Integer> answers, int
timeTakenSeconds) {
        Quiz quiz = quizRepository.findById(quizId)
            .orElseThrow(() -> new ResourceNotFoundException("Quiz not found with id: " + quizId));
        User student = userRepository.findById(studentId)
            .orElseThrow(() -> new ResourceNotFoundException("Student not found with id: " + studentId));

        int score = 0;
        int correctCount = 0;
        int incorrectCount = 0;
        int unansweredCount = 0;

        for (Question q : quiz.getQuestions()) {
            Integer selectedOption = answers.get(q.getId());
            if (selectedOption == null || selectedOption == -1) {
                unansweredCount++;
            } else if (selectedOption.equals(q.getAnswer())) {
                correctCount++;
                score += q.getMarks();
            } else {
                incorrectCount++;
            }
        }

        int totalMarks = quiz.getTotalMarks();
        double percentage = (totalMarks > 0) ? ((double) score / totalMarks) * 100.0 : 0.0;
        boolean passed = score >= quiz.getPassingMarks();

        TestAttempt attempt = new TestAttempt();
        attempt.setQuiz(quiz);
        attempt.setStudent(student);
        attempt.setScore(score);
        attempt.setTotalMarks(totalMarks);
        attempt.setPercentage(Math.round(percentage * 100.0) / 100.0);
        attempt.setPassed(passed);
        attempt.setCorrectCount(correctCount);
        attempt.setIncorrectCount(incorrectCount);
        attempt.setUnansweredCount(unansweredCount);
        attempt.setTimeTakenSeconds(timeTakenSeconds);
        attempt.setSubmittedAt(LocalDateTime.now());
        attempt.setAnswersJson(serializeAnswers(answers));
        attempt.setEmailStatus("PENDING");

        TestAttempt savedAttempt = attemptRepository.save(attempt);
        emailService.sendScorecardEmailAsync(savedAttempt);
        return savedAttempt;
    }
}
```

Technical Walkthrough: Evaluates submitted responses against relational MySQL records within a database transaction. It computes correct, incorrect, and unanswered counts, calculates total marks and percentage scores, evaluates passing status, commits the attempt record, and triggers asynchronous email dispatch.

Listing 6.2: Anti-Cheat DTO Sanitization Controller (QuizController.java)

backend/src/main/java/com/skillnest/controller/QuizController.java

```
@RestController
@RequestMapping("/api/quizzes")
@CrossOrigin(origins = "*")
public class QuizController {
    @Autowired
    private QuizService quizService;

    @GetMapping("/{id}/take")
    public ResponseEntity<TakeQuizDTO> getQuizForTestTaking(@PathVariable Long id) {
        Quiz quiz = quizService.getQuizById(id);
        if (!"PUBLISHED".equalsIgnoreCase(quiz.getStatus())) {
            return ResponseEntity.status(HttpStatus.FORBIDDEN).build();
        }

        List<TakeTestQuestionDTO> sanitizedQuestions = quiz.getQuestions().stream()
            .map(q -> new TakeTestQuestionDTO(
                q.getId(),
                q.getTitle(),
                q.getCategory(),
                q.getTopic(),
                q.getDifficulty(),
                q.getOptionA(),
                q.getOptionB(),
                q.getOptionC(),
                q.getOptionD(),
                q.getMarks()
                // Notice: q.getAnswer() and q.getExplanation() are omitted
            ))
            .collect(Collectors.toList());

        TakeQuizDTO takeQuizDTO = new TakeQuizDTO(
            quiz.getId(),
            quiz.getTitle(),
            quiz.getCategory(),
            quiz.getDuration(),
            quiz.getDescription(),
            quiz.getTotalMarks(),
            quiz.getPassingMarks(),
            sanitizedQuestions
        );

        return ResponseEntity.ok(takeQuizDTO);
    }
}
```

Technical Walkthrough: Enforces server-side anti-cheat protection. When a candidate launches a test, this endpoint maps questions to `TakeTestQuestionDTO` instances, omitting `answer` and `explanation` attributes from the JSON response to prevent client-side inspection leakage.

Listing 6.3: Responsive HTML Email Dispatcher (EmailService.java)

backend/src/main/java/com/skillnest/service/EmailService.java

```
@Service
public class EmailService {
    @Autowired(required = false)
    private JavaMailSender mailSender;
    @Value("${spring.mail.username:noreply@skillnest.com}")
    private String senderEmail;

    @Async
    public void sendScorecardEmailAsync(TestAttempt attempt) {
        if (mailSender == null) {
            attempt.setEmailStatus("NOT_CONFIGURED");
            return;
        }
        try {
            MimeMessage message = mailSender.createMimeMessage();
            MimeMessageHelper helper = new MimeMessageHelper(message, true, "UTF-8");
            helper.setFrom(senderEmail, "SkillNest Assessment Team");
            helper.setTo(attempt.getStudent().getEmail());
            helper.setSubject("SkillNest Assessment Scorecard: " + attempt.getQuiz().getTitle());

            String htmlContent = buildScorecardHtml(attempt);
            helper.setText(htmlContent, true);

            mailSender.send(message);
            attempt.setEmailStatus("SENT");
        } catch (Exception ex) {
            attempt.setEmailStatus("FAILED");
        }
    }
}
```

Technical Walkthrough: Asynchronously compiles and dispatches responsive multi-part MIME HTML scorecard emails to candidates without blocking HTTP response threads. If mail credentials are unconfigured, it records `NOT_CONFIGURED` status gracefully without raising unhandled runtime errors.

Listing 6.4: Cryptographic Password & CORS Security Configuration (SecurityConfig.java)

backend/src/main/java/com/skillnest/config/SecurityConfig.java

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public BCryptPasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(10);
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**", "/api/quizzes/**", "/api/questions/**",
"/api/attempts/**").permitAll()
                .anyRequest().authenticated()
            );
        return http.build();
    }
}
```

Technical Walkthrough: Registers a BCrypt encoder configured with an adaptive work factor of 10 rounds for one-way credential hashing. It establishes stateless session management and configures permissive CORS headers to facilitate local and distributed development between Vite and Spring Boot.

Listing 6.5: React Countdown Timer & Auto-Submit Hook (TakeQuiz.jsx)

frontend/src/pages/TakeQuiz.jsx

```
useEffect(() => {
  if (timeLeft <= 0 && !isSubmitted) {
    handleAutoSubmit();
    return;
  }
  const timer = setInterval(() => {
    setTimeLeft(prev => {
      if (prev <= 1) {
        clearInterval(timer);
        handleAutoSubmit();
        return 0;
      }
      return prev - 1;
    });
  }, 1000);

  return () => clearInterval(timer);
}, [timeLeft, isSubmitted]);

const handleAutoSubmit = async () => {
  setIsSubmitting(true);
  try {
    const timeSpent = Math.max(1, (quiz.duration * 60) - timeLeft);
    const response = await axios.post(`/api/quizzes/${id}/submit`, {
      studentId: currentUser.id,
      answers: selectedAnswers,
      timeTakenSeconds: timeSpent
    });
    setResult(response.data);
    setIsSubmitted(true);
  } catch (error) {
    console.error("Auto submission failed:", error);
  } finally {
    setIsSubmitting(false);
  }
};
```

Technical Walkthrough: Employs a React `useEffect` lifecycle hook to operate an active 1-second interval countdown timer. When the remaining time hits zero, it cancels the interval and invokes `handleAutoSubmit` to serialize candidate answers and post them to the backend grading engine.

7. LIMITATIONS

While SkillNest provides an enterprise-aligned assessment architecture, academic honesty requires documenting current boundaries:

- **Authentication Session Scope:** The platform utilizes BCrypt credential validation with client-stored user state in `localStorage`. It does not currently implement asymmetric JWT tokens or OAuth2/OpenID Connect social sign-in.
- **Static Analytics Cockpits:** Candidate dashboards calculate metrics directly from completed attempts, while comprehensive multi-dimensional cohort distributions and company-specific cutoffs are static mock views.
- **SMTP Infrastructure Dependency:** Automated email dispatch depends on external SMTP servers. When running in offline or air-gapped laboratory environments without internet access, email dispatch is automatically bypassed.
- **Evaluation Format:** Assessments are restricted to multiple-choice question formats. The platform does not currently incorporate an embedded sandboxed code execution engine (such as Judge0 or Docker-based code execution).

8. FUTURE SCOPE

SkillNest is architected for modular expansion across multiple enhancement vectors:

- **JWT Authentication & OAuth2 Integration:** Upgrade authentication to asymmetric RSA-signed JSON Web Tokens with refresh token rotation and institutional Google/GitHub SSO login.
- **Online Code Compiler Sandbox:** Integrate an isolated remote code execution engine (Judge0 or Docker sandboxing) enabling candidates to solve algorithmic problems with multi-language execution and test case validation.
- **AI-Powered Question Synthesis:** Incorporate the Google Gemini API to dynamically generate novel interview questions, generate customized hints, and analyze code explanations.
- **Automated Resume & ATS Score Calculator:** Develop a placement resume audit engine that evaluates candidate resumes against target job descriptions and extracts skill gaps.
- **Proctoring & Anti-Tab Switching Monitor:** Introduce WebRTC-based webcam proctoring, face detection, audio noise classification, and tab-switch monitoring to uphold exam integrity during remote campus placements.

9. CONCLUSION

The **SkillNest: Full-Stack Technical Interview & Placement Preparation Platform** was successfully designed, developed, and verified as an MCA Mini Project during the academic year 2026-2027.

The project successfully addresses the fragmentation in campus placement preparation by uniting a curated 9-domain technical repository (Java, Python, C++, HTML, CSS, JavaScript, React, Node.js, SQL) with an anti-cheat assessment sandbox. Key milestones achieved include:

- Developed a decoupled service-oriented architecture integrating a React 19 single-page application with a Spring Boot 4.1.1 RESTful backend and MySQL 8.0 database.
- Seeded and organized 108+ vetted technical questions with comprehensive technical explanations.
- Engineered a server-side anti-cheat sanitization protocol via `TakeTestQuestionDTO` that eliminates answer leakage during active tests.
- Implemented an active test HUD equipped with real-time countdown timers, color-coded question navigation palettes, and auto-submission mechanisms.
- Delivered an automated server-side evaluation engine calculating scores, percentages, and pass/fail thresholds atomically.
- Built asynchronous multi-part MIME HTML scorecard dispatch via Spring Boot's `JavaMailSender` with graceful fallback handling.

SkillNest provides a robust, extensible foundation for technical interview readiness and placement calibration for master's degree computer application students.

10. REFERENCES

1. C. Walls, *Spring in Action*, 6th ed. Shelter Island, NY: Manning Publications, 2022.
2. C. Bauer, G. King, and G. Gregory, *Java Persistence with Hibernate*, 2nd ed. Shelter Island, NY: Manning Publications, 2016.
3. R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Boston, MA: Prentice Hall, 2008.
4. M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Boston, MA: Addison-Wesley, 2018.
5. Spring Boot Reference Documentation, "Spring Boot Framework Documentation (v4.x/3.x)," VMware Tanzu, 2024. [Online]. Available: https://docs.spring.io/spring-boot/docs/
6. React Documentation, "React 19: The Library for Web and Native User Interfaces," Meta Open Source, 2024. [Online]. Available: https://react.dev/
7. Oracle Corporation, "MySQL 8.0 Reference Manual," Oracle Corporation, 2024. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/
8. R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.
9. Mozilla Developer Network (MDN), "Web Security, Cross-Origin Resource Sharing (CORS), and Local Storage," Mozilla Foundation, 2024. [Online]. Available: https://developer.mozilla.org/
10. Vite Documentation, "Vite: Next Generation Frontend Tooling," 2024. [Online]. Available: https://vitejs.dev/